

Reviewer Pack — Minimal Order of Braided Tensor Product Modules and Communic...

Entry 552fe11bad4a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 15:21:47 UTC

Minimal Order of Braided Tensor Product Modules and Communication Complexity Rank Variance Ratio

Entry ID: 552fe11bad4a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 15:21:47 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Braided Categories) **Field B** (complexity object): Communication Complexity (Matrix Rank Variance)

Statement:

For any given n -ary Boolean function f , the minimal order of a braided tensor product module M that computes f is linearly correlated with the communication complexity rank variance ratio $R(f)$ of f , such that $\log_2 |M| = \Theta(\log(n) * R(f))$.

Rationale (proposer's reasoning):

Braided categories offer a categorical framework for studying tensor products and their computational properties. This conjecture suggests that the algebraic structure of braided modules could provide insights into communication complexity by quantifying the relationship between module order and communication complexity measures.

Taxonomy category: Braided Categories × Communication Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d24c586634ffc7dd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all n -ary Boolean functions f , the computed logarithm base 2 of the order of the braided tensor product module M ($|M|$) has a significant positive correlation with $\log(n) * R(f)$ using ANOVA test on 30 random seeds, where $R(f)$ is the communication complexity rank variance ratio and $|M|$ is measured by the aggregate mean across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Braided Tensor Product Modules' AND 'Communication Complexity Rank Variance Ratio' - 'Minimal order braided categories' AND 'communication complexity rank variance' - 'Category theory' AND 'Boolean function communication complexity'

Top relevant hits considered: - [<http://arxiv.org/abs/1607.03608v3>] On the tensor product of linear sites and Grothendieck categories - [<http://arxiv.org/abs/1808.10521v2>] Efficient quantum tensor product expanders and unitary t-designs via the zigzag product - [<http://arxiv.org/abs/2207.05809v2>] On the stability of tensor product of representations of classical groups - [<http://arxiv.org/abs/1204.4836v5>] Rank 4 Premodular Categories - [<http://arxiv.org/abs/1705.04238v2>] Braided module categories via quantum symmetric pairs - [<http://arxiv.org/abs/1805.01774v2>] A Tangent Category Alternative to the Faà di Bruno Construction - [<http://arxiv.org/abs/1802.09555v2>] Involutive categories, colored $\ast$ -operads and quantum field theory - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank_variance_ratio(f):
        n = len(f)
        m = 2**n
        rank = 0
        for i in range(m):
            if f[i] == 1:
                rank += 1
        return rank / m

    def construct_braided_tensor_product_module(n, f):
        # Construct a simple module based on the function
```

```

    M = [f]
    return M

n_values = [5, 10, 15, 20, 30, 40]
metric_values = []

for n in n_values:
    f = generate_random_boolean_function(n)
    R_f = communication_complexity_rank_variance_ratio(f)
    M = construct_braided_tensor_product_module(n, f)
    log_M = math.log2(len(M))
    log_nR_f = math.log2(n) * R_f
    metric_values.append((log_M, log_nR_f))

if len(metric_values) < 30:
    return {
        "metric_name": "log|M| vs. log(n) * R(f)",
        "metric_value": None,
        "instances_tested": len(metric_values),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_log_M = sum(x[0] for x in metric_values) / len(metric_values)
mean_log_nR_f = sum(x[1] for x in metric_values) / len(metric_values)
correlation = 0
for log_M, log_nR_f in metric_values:
    correlation += (log_M - mean_log_M) * (log_nR_f - mean_log_nR_f)
correlation /= len(metric_values) * math.sqrt(sum((x[0] - mean_log_M)**2 for x in metric_values))

return {
    "metric_name": "log|M| vs. log(n) * R(f)",
    "metric_value": correlation,
    "instances_tested": len(metric_values),
    "n_max": max(n_values),
    "conjecture_holds": abs(correlation) > 0.5, # Threshold for significance
    "counterexample": ""
}

if __name__ == "__main__":
    if not sys.argv[1:]:
        seeds = [2*i + 7 for i in range(30)] # First 30 prime numbers
    else:
        seeds = list(map(int, sys.argv[1:]))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) /
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results if r["metric_value"] is not None) /
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"not_supported\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a3bb67f.py", line 83, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a3bb67f.py", line 43, in run_trial
    R_f = communication_complexity_rank_variance_ratio(f)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5a3bb67f.py", line 29, in communication_complexity_rank_variance_ratio
    if f[i] == 1:
       ~^^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which prevents us from evaluating the conjecture's support conditions. | next: Re-run the test without errors to verify the conjecture's support conditions.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13149
2	propose	ollama_remote	glm4:latest	0	0	13201
3	propose	ollama_remote	glm4:latest	0	0	16049
4	preregistration	ollama_remote	glm4:latest	0	0	9771
5	novelty	ollama_remote	glm4:latest	0	0	8493
6	novelty	ollama_remote	glm4:latest	0	0	10111

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	42746
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9782
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10726
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10251
11	judge	ollama_remote	glm4:latest	0	0	40019

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 184299 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/552fe11bad4a.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/552fe11bad4a.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/552fe11bad4a.tar.gz* (if generated)